



CNNs, Object Detection & Segmentation

AIML Program | Industry Practitioner Track

September 2026

What you'll be able to do by the end

Five outcomes, from the neuron to industrial computer vision.

01



Explain the CNN pipeline

Trace an image through the CNN pipeline and why each step matters.

02



Tell the three tasks apart

Distinguish classification, detection and segmentation.

03



Read detection & mask metrics

Interpret IoU, mAP, Dice and mIoU.

04



Build & train a model

Build a CNN and YOLO-based model in Keras.

05



Map it to a plant problem

Apply the three tasks to a mineral-processing use case.

Why classical image processing hits a wall

The old way: hand-crafted features



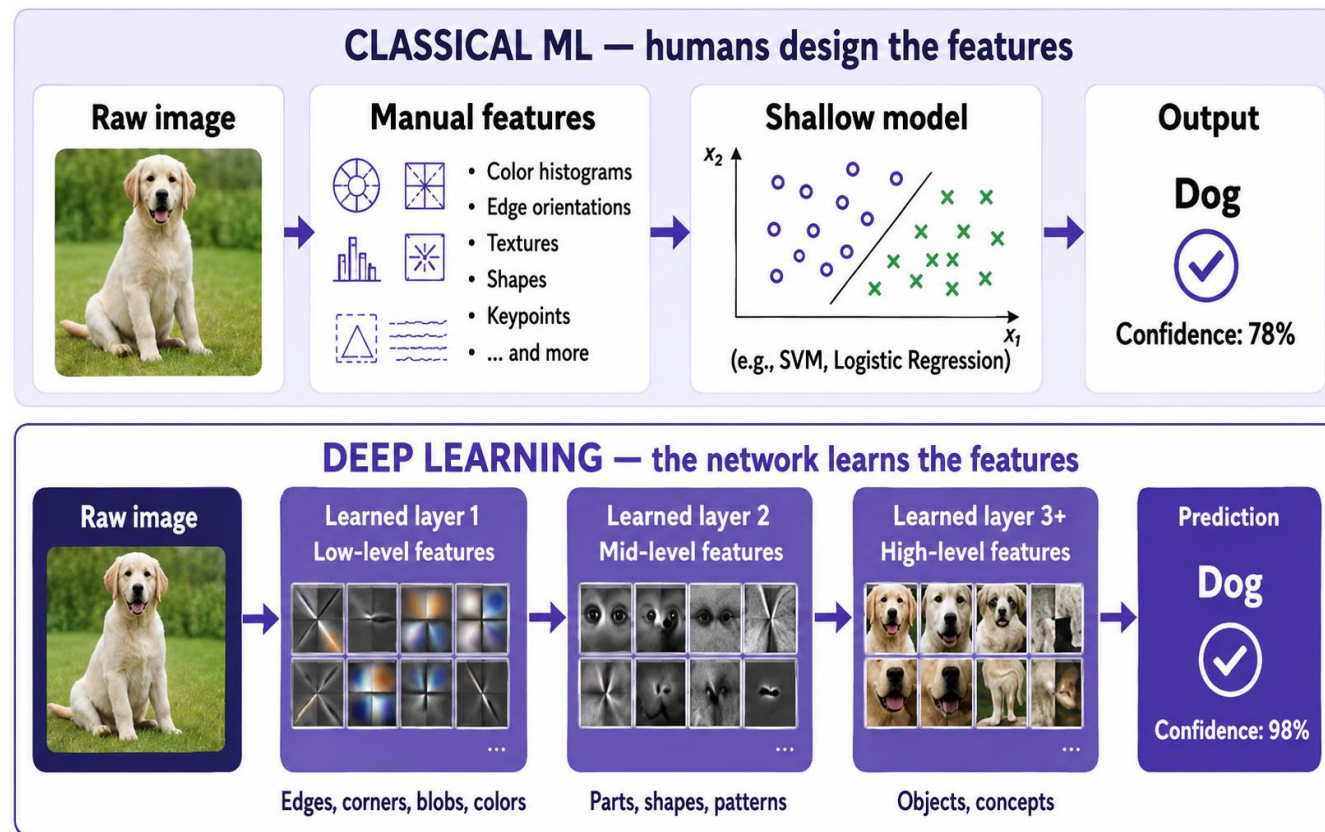
Edges, corners, SIFT/HOG are designed by humans — expensive, brittle, domain-specific.



Every new task, camera or lighting means re-engineering the features from scratch.



Performance plateaus: more data barely helps once the features are fixed.



The core promise: a CNN learns useful visual representations directly from raw pixels — no hand-crafted features required.

Three questions, three tasks

Before the deep dive: each computer-vision task asks a harder question and returns a richer output.

01

Classification

What is in the image?

Prediction

Cat

Confidence

92%

One label for the whole image

cat

02

Object Detection

What objects, and where?

A box + label + score per object

3 boxes

03

Segmentation

Which pixels belong to what?

A per-pixel class / object mask

pixel mask

Why a plain dense network fails on images

DENSE: flatten & fully-connect



Too many parameters

A dense layer on a $224 \times 224 \times 3$ image needs ~150 M weights in the first layer alone — unmanageable.



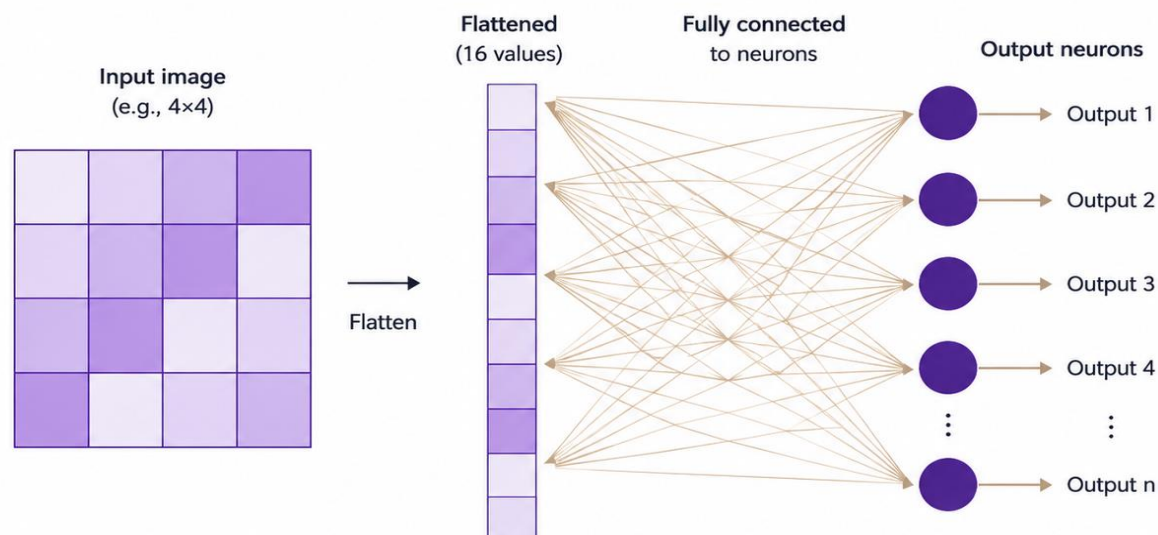
Ignores spatial structure

Flattening an image throws away the fact that nearby pixels are related.



Not translation-invariant

A cat top-left looks like a totally different input from the same cat bottom-right.



~150 M weights in ONE layer

For a $224 \times 224 \times 3$ image $\rightarrow$ **150,528** inputs.
Connecting to just 1,000 neurons already
requires ~150 million weights!

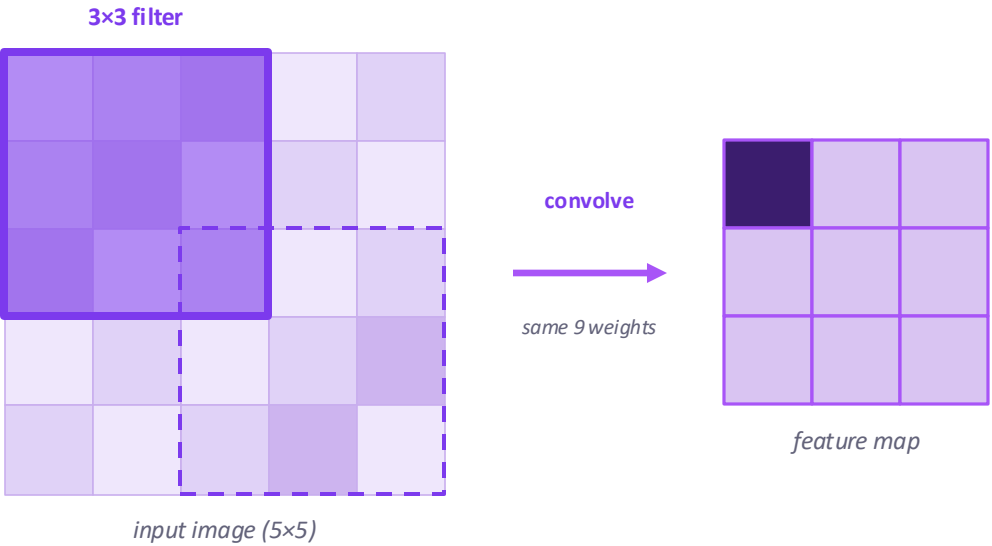
Example

- Inputs = $224 \times 224 \times 3 = 150,528$
- Neurons = 1,000 (example)

Weights $\approx 150,528 \times 1,000 = 150,528,000$

The CNN idea: local filters, shared weights

Instead of wiring every pixel to every neuron, a CNN slides one small filter across the image — reusing the same weights everywhere.



The payoff — far fewer weights

A dense layer needs a weight for every input, while a CNN reuses a small filter.

Layer Type	Weights
Dense, fully-connected 224x224x3 → 1,000 neurons Each input has its own set of weights.	≈150 M weights
CNN, one shared 3x3 filter reused at every position Same filter scans the whole image.	27 weights per filter (3x3x3)

Same job. Very different number of weights.

The same 27 weights are reused as the filter slides across the entire 224 × 224 image.



Local receptive fields

Each neuron looks at only a small patch of the image — not the whole thing.



Weight sharing

The same small filter slides across the image, so a handful of weights do all the work.



Translation invariance

Because the filter is reused everywhere, a pattern is found wherever it appears.

The artificial neuron — where it all starts



Weighted sum (logit)

Combine inputs using learned weights and a bias.

$$z = w \cdot x + b$$



Activation

Apply a non-linearity $\varphi()$ to produce the output **a**.

$$a = \varphi(z)$$

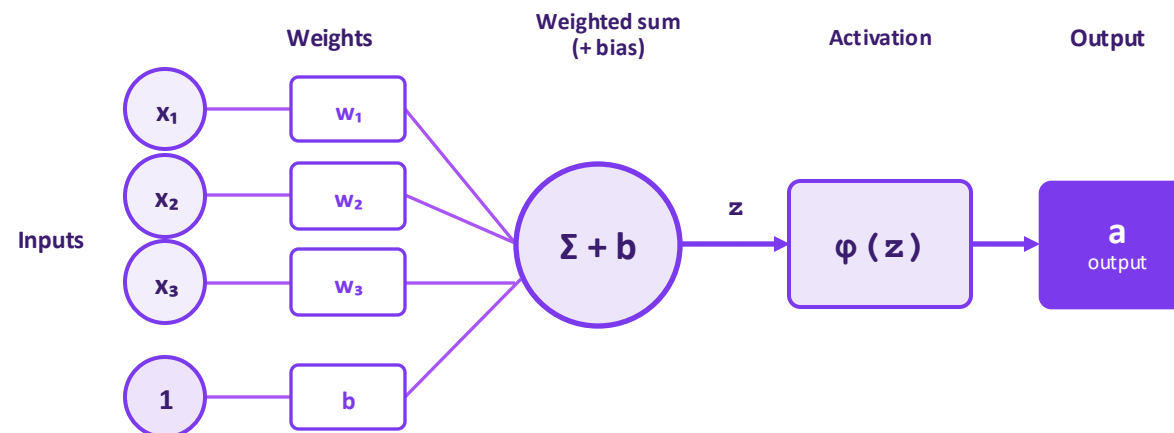


Learning = tuning

Training adjusts weights and bias to reduce error.

Anatomy of a neuron

Inputs are weighted, summed with a bias, passed through an activation, and produce one output.



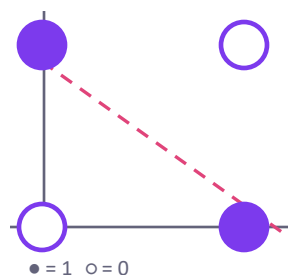
A **perceptron** is the simplest type of artificial neural network and functions as a linear binary classifier. It takes multiple numerical inputs, multiplies each by a specific weight, adds them together, and passes the total sum through an activation function to output a single binary decision (0 or 1).

From one neuron to a deep network

Depth unlocks complexity

- One neuron draws only a straight line — it can't even solve XOR.
- Dense layers: each neuron connects to every neuron in the next.
- Hidden layers compose simple features into richer ones.
- Non-linear activations are what make depth worthwhile.

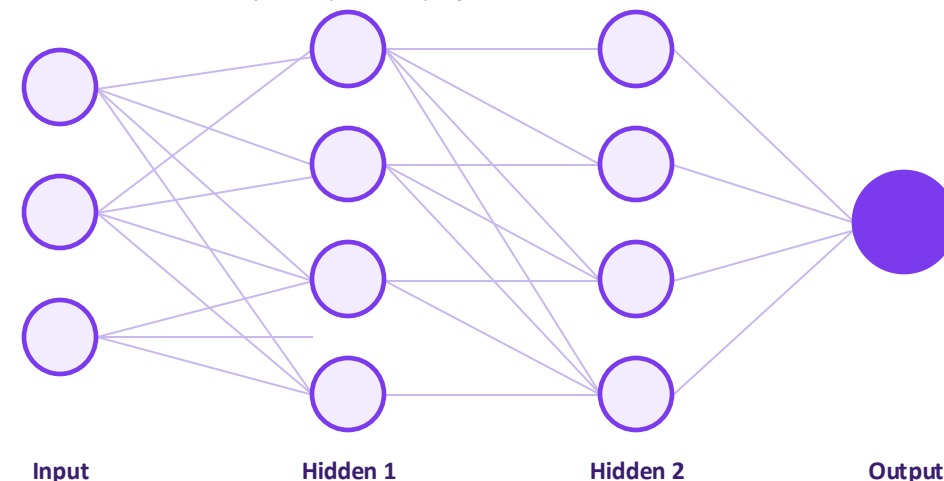
The XOR problem



One straight line can't separate XOR — hidden layers bend the boundary.

Stack them into layers

Each hidden layer composes simple features into richer ones.



Pick an activation to match the job

ReLU

$$\max(0, z)$$



Default for hidden layers — fast, no saturation for $z > 0$.

Sigmoid

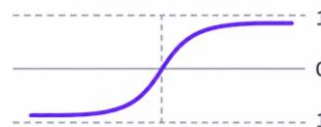
$$\sigma(z) \in (0, 1)$$



Squashes to a probability — use for binary output.

Tanh

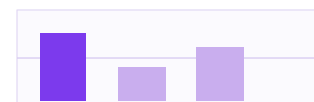
$$\tanh(z) \in (-1, 1)$$



Zero-centered output — useful when hidden-layer activations benefit from both positive and negative values.

Softmax

$$\sum \text{outputs} = 1$$

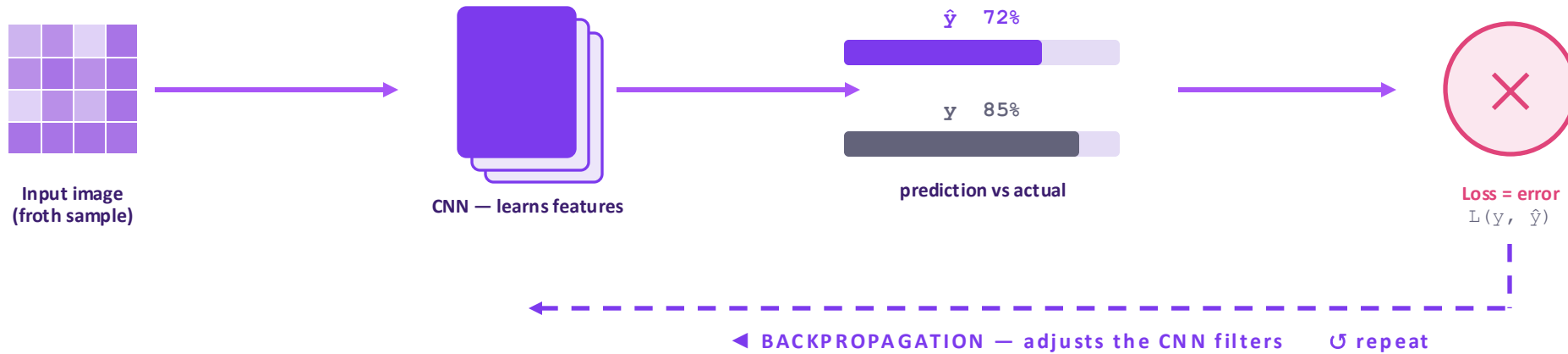


Turns scores into class probabilities — multi-class output.

Forward pass → loss → backpropagation

A CNN learns by trial and error: predict, measure how wrong it was, then adjust — using a froth-quality example (refer to Slide-28).

FORWARD PASS ▶



1 · Forward

Make a prediction.

What did I predict?



2 · Loss

Measure the error.

How wrong was I?



3 · Backprop

Find the weights to blame.

What should change?



4 · Update

Nudge each weight.

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \partial L / \partial w$$

Training = predict → measure error → learn from it → update → ↺ repeat

The convolution operation, step by step

A small filter slides over the image; at each position it multiplies overlapping values and sums them into one output number.

Input (5×5)

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Kernel (3×3)

1	0	1
0	1	0
1	0	1

Feature map (3×3)

4	3	4
2	4	3
2	3	4

First position → 4

multiply the overlapping cells, then add:

$$\begin{aligned}
 &(1 \times 1) + (1 \times 0) + (1 \times 1) \\
 &+ (0 \times 0) + (1 \times 1) + (1 \times 0) \\
 &+ (0 \times 1) + (0 \times 0) + (1 \times 1) = 4
 \end{aligned}$$

the filter slides across the image → one value per position

Each output = Σ (kernel × overlapping patch). Each position → multiply + sum → one feature value.

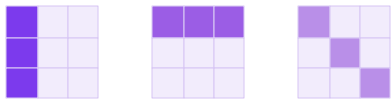
The filter's weights are learned to detect useful patterns — edges, textures, shapes, or more abstract features in deeper layers.

One layer, many filters — the knobs that matter



Multiple filters

Each filter learns a different pattern and makes its own feature map. Stacking them gives the layer's depth.



each filter → its own feature map



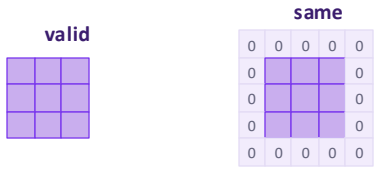
Stride

How far the filter jumps each step. Stride 2 halves the output — cheap downsampling.



Padding

"same" preserves spatial size; "valid" uses no padding and shrinks the output.*

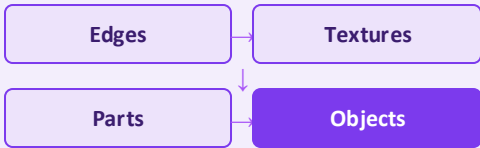


* when stride = 1



Depth stacks up

Early layers find simple patterns; deeper layers combine them into complex ones.



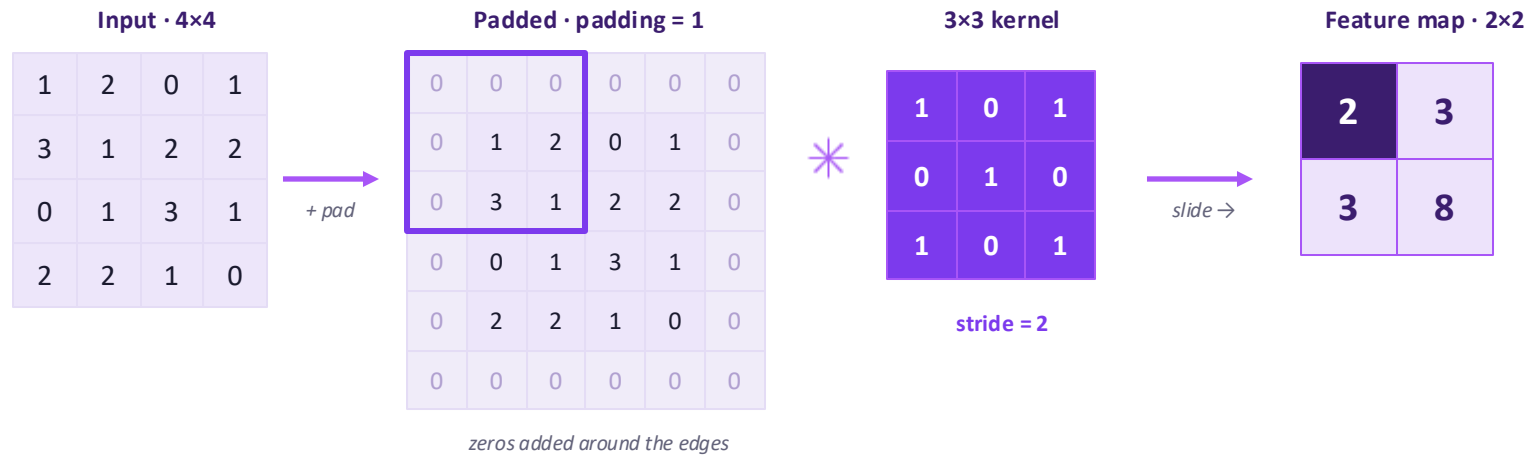
$$\text{Output} = \lfloor (n - k + 2p) / s \rfloor + 1$$

n input size **k** kernel/filter size **p** padding **s** stride

Know this — and you can size any convolution layer.

Convolution in action: filter → slide → feature map

A 3×3 filter with stride 2 and padding 1.



First position → 2

multiply the top-left window (with padding) by the kernel, then add:

$$\begin{aligned}
 &(0 \cdot 1) + (0 \cdot 0) + (0 \cdot 1) \\
 &+ (0 \cdot 0) + (1 \cdot 1) + (2 \cdot 0) \\
 &+ (0 \cdot 1) + (3 \cdot 0) + (1 \cdot 1) \\
 &= 2
 \end{aligned}$$

$$\text{Output size} = \lfloor (n - k + 2p) / s \rfloor + 1 \quad \lfloor (4 - 3 + 2 \times 1) / 2 \rfloor + 1 = \lfloor 3/2 \rfloor + 1 = 2 \rightarrow 2 \times 2 \text{ feature map}$$

Takeaway: padding protects border information; stride controls how far the filter moves.

Pooling — shrink, summarize, stay robust

What pooling does

A small window slides across the feature map and summarises each patch into one value.

Max pooling → strongest activation

Average pooling → average activation

Why it's worth it

- ✓ **Smaller feature maps** → less compute & memory
- ✓ **Small shifts tolerated** → more robust predictions
- ✓ **Larger receptive field** → wider context for later layers

2×2 max pooling, stride 2

3	1	2	4
2	0	1	3
1	5	4	2
0	2	3	6

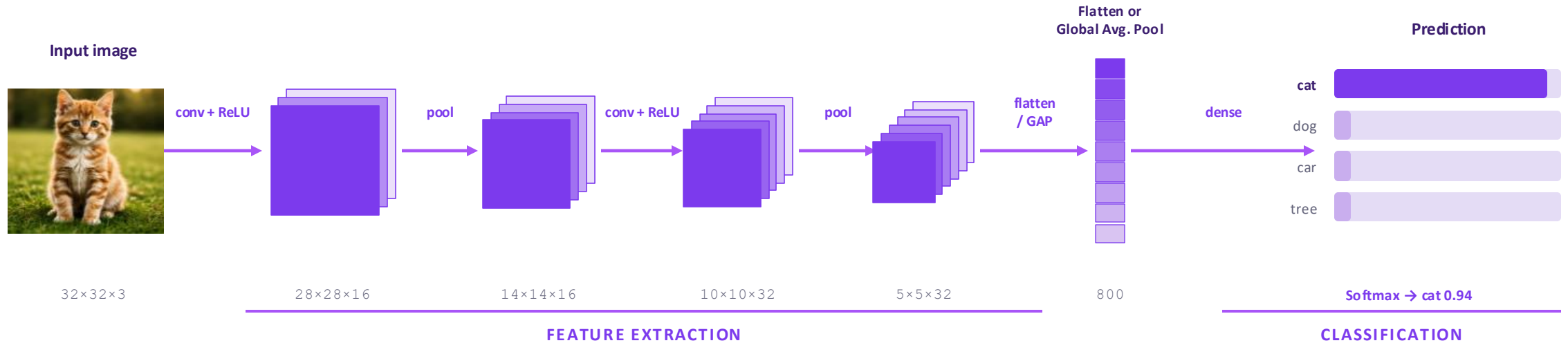
→
max

3	4
5	6

each 2×2 patch → one value

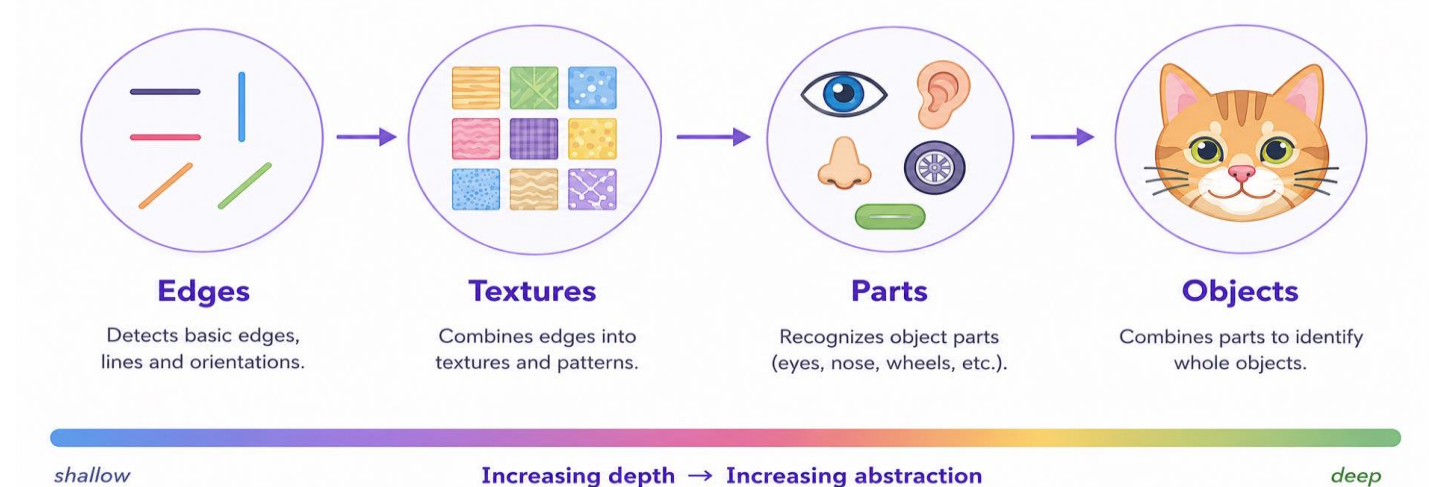
Each 2×2 patch → its maximum value. With stride 2, spatial size is halved.

The anatomy of a typical CNN



As data flows right, width & height shrink while depth (number of channels) grows.

The feature hierarchy — why depth works



From pixels to a digit: MNIST end-to-end

A small CNN learns handwritten digits: pixels → stroke features → digit parts → one label.



Intuition to carry forward

A CNN isn't told what an edge or loop is. During training, it learns features that help minimize prediction loss. *The same idea scales from handwritten digits to defects on a conveyor belt.*

Building a CNN in Keras

cnn.py

```
model = keras.Sequential([
    layers.Conv2D(32, 3, activation="relu",
                  padding="same",
                  input_shape=[28, 28, 1]),
    layers.MaxPooling2D(2),           # Reduce spatial size
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(2),           # Reduce spatial size
    layers.Flatten(),                 # Convert feature maps to 1D
    layers.Dense(128, activation="relu"), # Learn high-level features
    layers.Dropout(0.5),              # Reduce overfitting
    layers.Dense(10, activation="softmax"),
])
```

Conv → Pool blocks

Filters double (32→64) as spatial size halves — a classic pyramid.

padding="same"

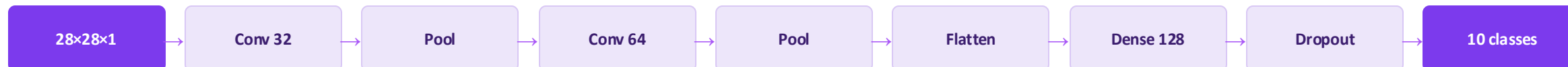
Keeps width/height so only pooling changes the size.

Dropout (0.5)

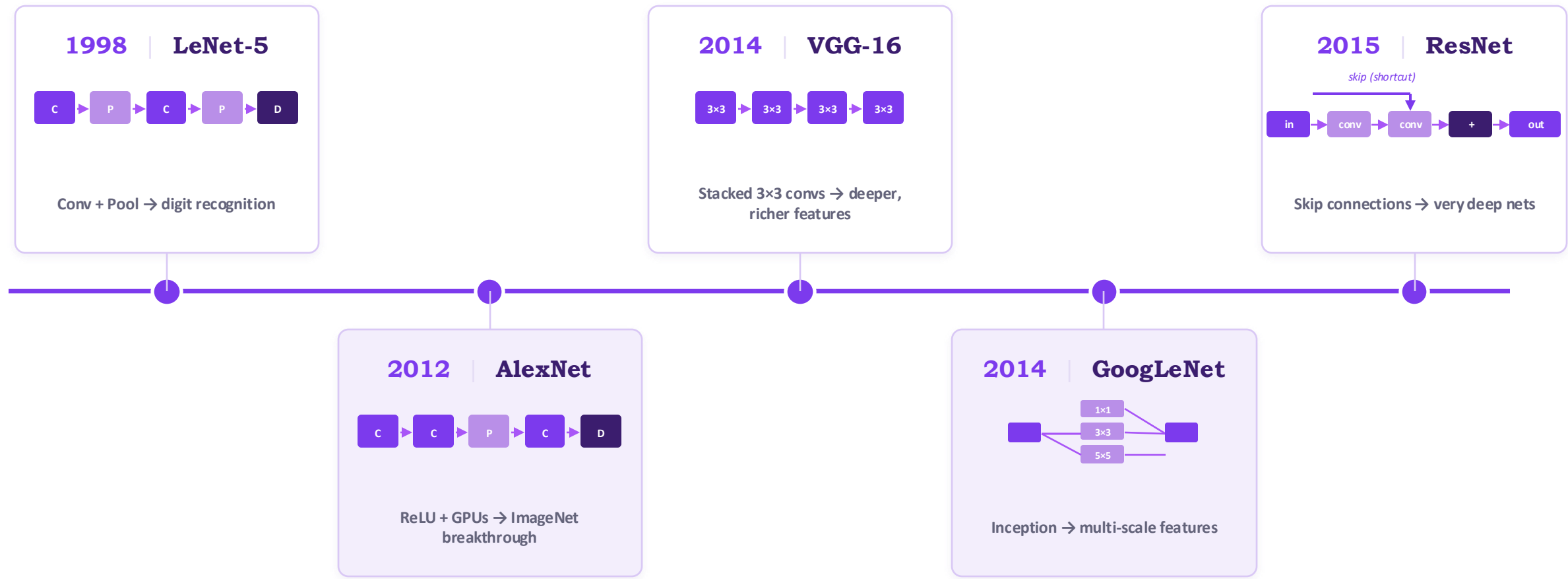
Regularises the dense head against overfitting.

Softmax (10)

One probability per class; pair with categorical cross-entropy.



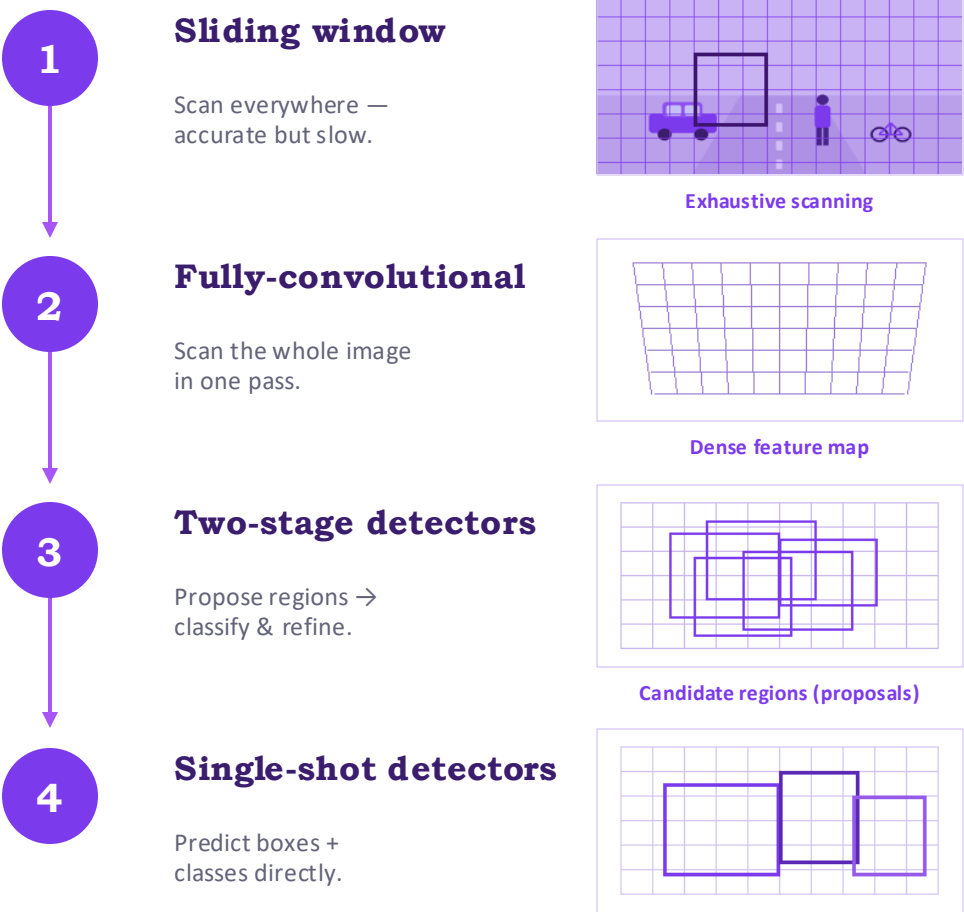
Landmark CNN Architectures



Evolution: deeper networks + smarter connections → richer features → stronger CNN backbones.

Object Detection: what & where, at once

From brute-force scanning → smarter prediction



Detection output = boxes + labels + scores

Box (where)
Location of the object

Label (what)
Object class (e.g., car, person)

Score (how confident)
Model's confidence (0–1)

Box

 +

Label

 +

Score

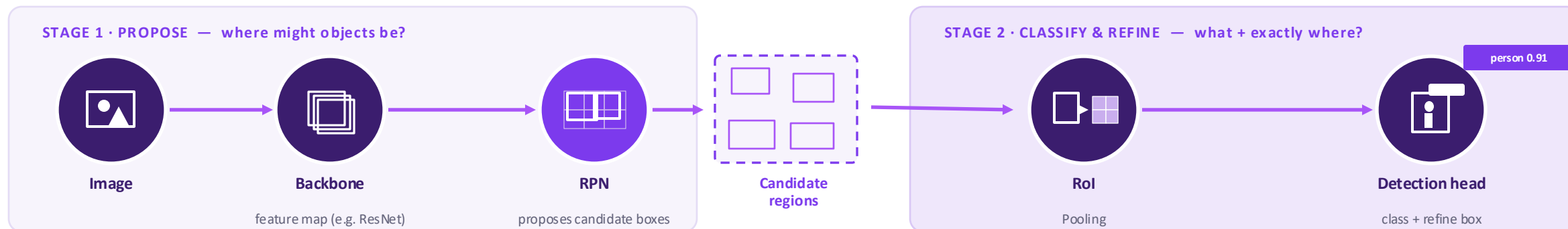
 →

Detection
(What + Where + How confident)

	Two-stage	Single-shot
Example	Faster R-CNN	YOLO / SSD
Speed	Slower	Faster
Accuracy	High	High

Two-stage detectors: propose, then classify

Stage 1 proposes where objects might be; stage 2 classifies each region and sharpens its box.



The R-CNN lineage

- 1 **R-CNN** (2014)
Propose regions → CNN per region → accurate, slow
- 2 **Fast R-CNN** (2015)
CNN once → RoI pooling → faster
- 3 **Faster R-CNN** (2015)
RPN learns proposals → end-to-end

Why two-stage detectors still matter

- ✓ Often strong on small, dense or overlapping objects
- ✓ Extends to instance segmentation (Mask R-CNN)
- ✓ Backbone-agnostic — ResNet, ResNeXt, Swin
- ! **Trade-off:** often more accurate, but slower than single-shot

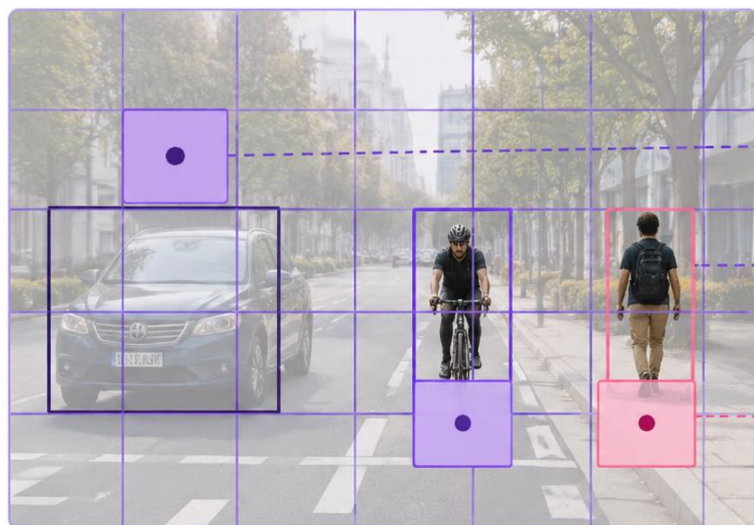


Takeaway: two-stage detectors first find promising regions (**where**), then identify and refine them (**what + exactly where**).

One-stage detectors: real-time in a single pass

Skip region proposals — predict boxes + classes directly in one pass.

YOLO idea: one grid, one pass



Each grid cell predicts:

box (x,y,w,h)
class car
conf 0.97

box (x,y,w,h)
class bicycle
conf 0.92

box (x,y,w,h)
class person
conf 0.89

(Other cells predict background or low-confidence objects)



Image is divided into an $S \times S$ grid.



Each cell is responsible for predictions whose center falls in that cell.



The cell predicts box (x,y,w,h), class, and confidence.



Boxes can overlap across cells, allowing detection of multiple objects.

Image → grid → each cell predicts boxes + class + confidence — all at once.



The YOLO family

v1 introduced the grid; later versions added multi-scale prediction, then anchor-free heads. Modern Ultralytics releases are the real-time default.



SSD

Predicts from several feature-map scales in one pass, so both small and large objects are caught without a second stage.



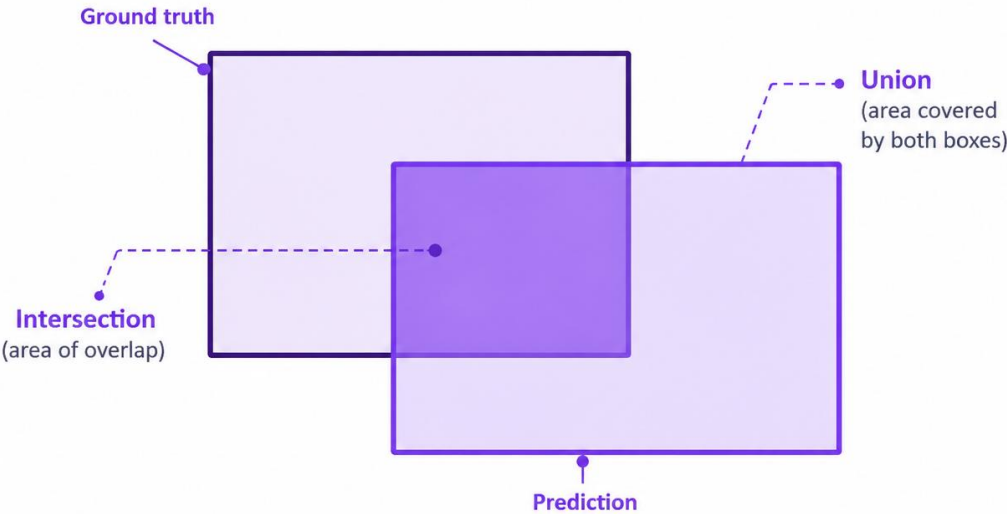
RetinaNet + Focal Loss

One-stage models see mostly background; focal loss down-weights easy negatives so rare objects still train — matching two-stage accuracy.

Key idea: one network → one pass → boxes + classes → fast inference.

IoU & NMS: measure overlap, keep the best box

Intersection over Union (IoU)



$$\text{IoU} = \frac{\text{Intersection area (overlap)}}{\text{Union area (combined area)}} = 0 \text{ to } 1$$

✓ 1 = perfect overlap

✗ 0 = no overlap

i A prediction counts as correct when its IoU with a true box clears a threshold (e.g., 0.5).



IoU threshold

A detection counts as a match when IoU crosses a chosen threshold (e.g. 0.5).

Non-Max Suppression (NMS)

Keep the highest-scoring box; remove overlapping duplicates.

Box 1		0.92	✓ keep
Box 2		0.81	✗ drop
Box 3		0.64	✗ drop



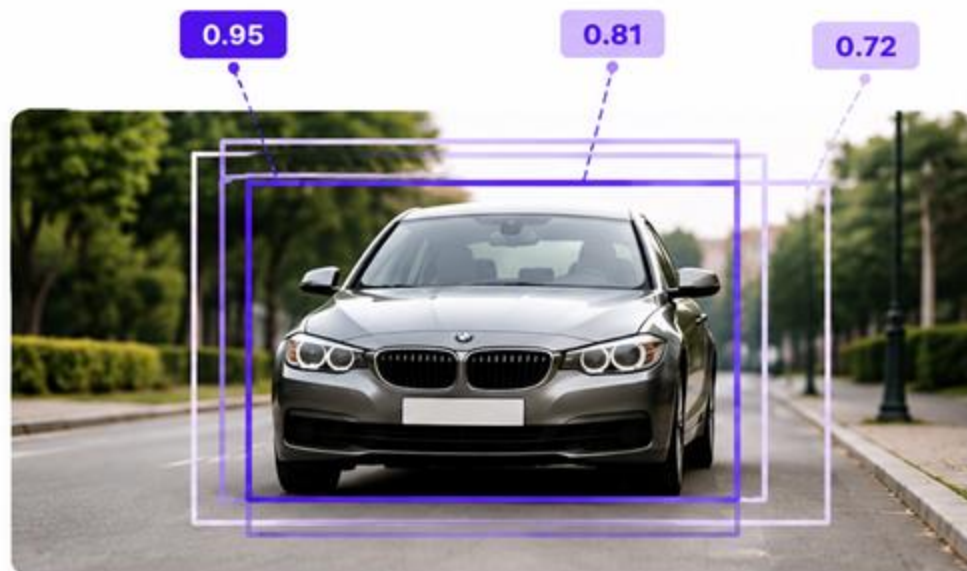
mAP (mean Average Precision)

Summarises detection performance across classes and recall levels. Higher is better.

Non-Max Suppression, visually

A detector fires many overlapping boxes for one object. NMS keeps the best and removes the duplicates.

BEFORE — many candidate boxes



Multiple predictions for the same object

Scores (confidence): **0.95** • **0.81** • **0.72**



NMS:

keep best →
remove
overlapping
duplicates

AFTER — NMS keeps the top box



Only the highest-scoring box remains

All other overlapping boxes are suppressed.

Semantic Segmentation: label every pixel

The goal

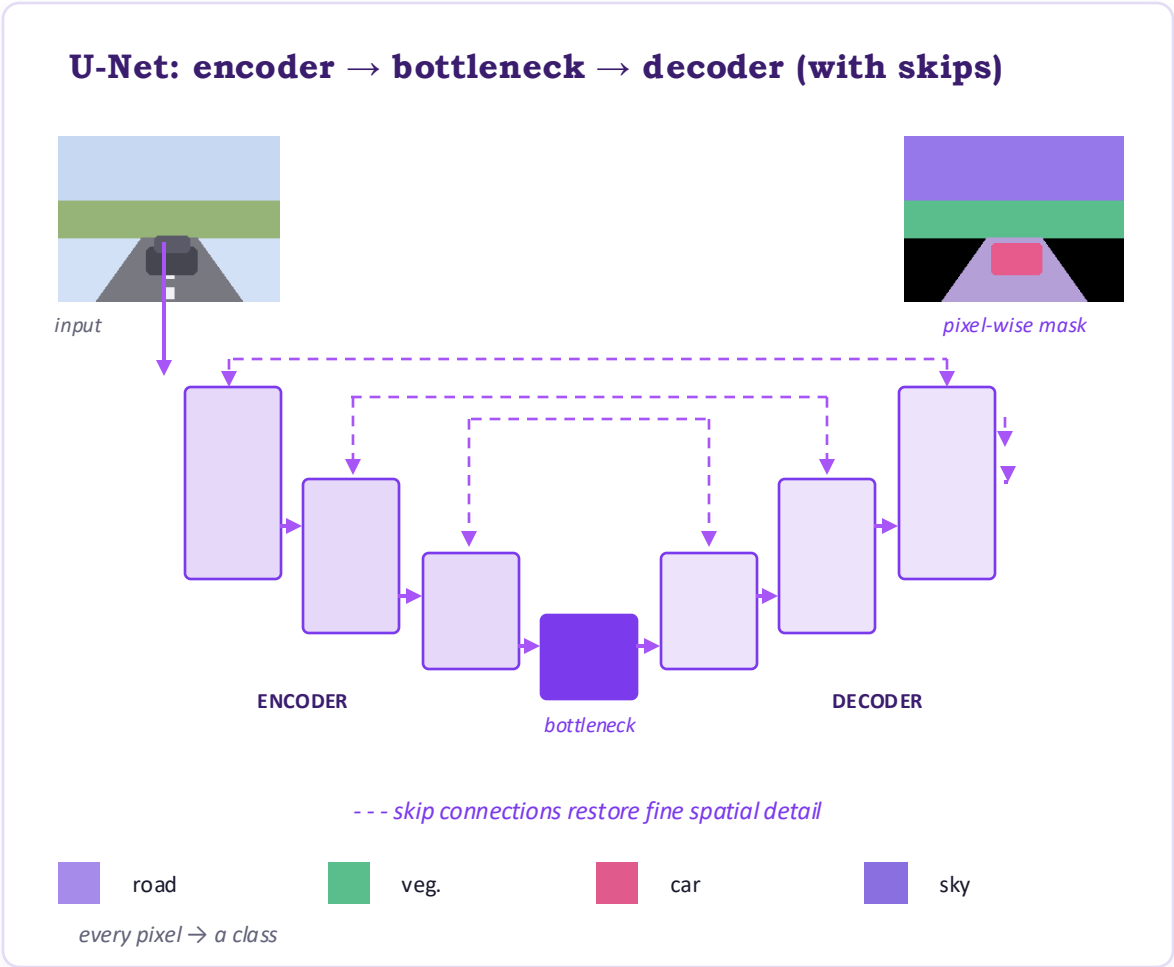
Assign a class to every pixel — a dense prediction map.

Why it is hard

Pooling captures what but loses where — the decoder must recover fine spatial detail.

Leading architectures

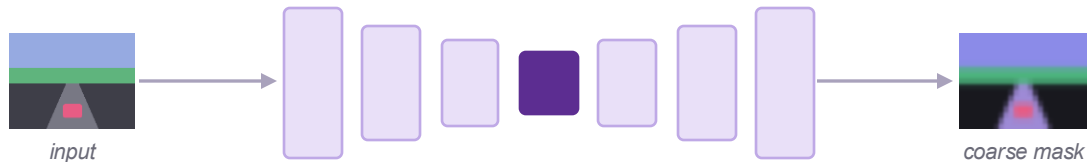
FCN	End-to-end fully convolutional
U-Net	Skip connections restore detail
DeepLab	Dilated convs + ASPP
PSPNet	Pyramid pooling adds global context



Why U-Net works so well for Segmentation

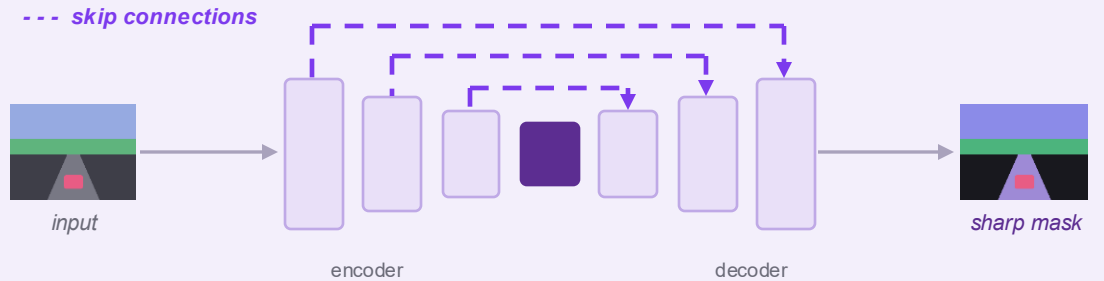
It combines “what” the image contains with “where” it is located.

Without skips



- Pooling loses fine spatial detail
- Decoder must reconstruct boundaries
- Small objects and edges are harder to recover

With U-Net skip connections



- Preserves fine spatial details
- Recovers accurate object boundaries
- Combines high-level context + low-level features

Key idea

Encoder → understands WHAT is present

Skip connections → preserve WHERE it is

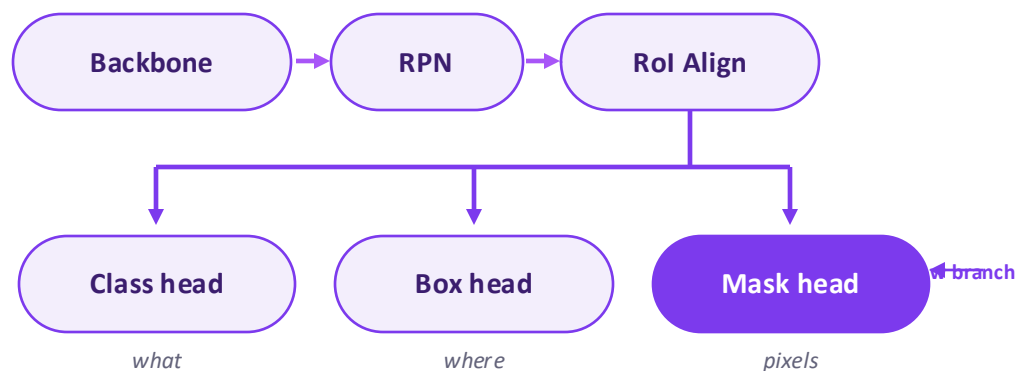
Decoder → produces a pixel-level mask

U-Net is effective because segmentation needs both context and precise localization.

Instance & Panoptic segmentation: separating objects

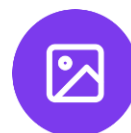
Semantic labels pixels; instance segmentation separates objects; panoptic combines both.

Mask R-CNN = Faster R-CNN + a mask branch



A small FCN mask head runs on every region — RoI Align keeps the pixels precisely aligned.

Three ways to segment



Semantic

Every pixel gets a class; touching objects can merge.



Instance

Each object gets its own mask + ID — two cars stay separate.



Panoptic

Every pixel gets a class + instance ID — things and stuff together.

Semantic vs. Instance vs. Panoptic Segmentation

Same scene, different segmentation outputs.

Input image



The original street scene (RGB camera input).

Semantic segmentation



Every pixel gets a class label. Objects of the same class merge.

Instance segmentation



Each object gets its own mask + ID. Same-class objects stay separate.

Panoptic segmentation



Every pixel gets a class label; “things” also get an instance ID.

Semantic = What • Instance = Which object • Panoptic = Both

Semantic classes

- Road
- Car
- Vegetation
- Building
- Sky

Instance IDs

- Car 1
- Car 2
- Car 3
- Bus 1

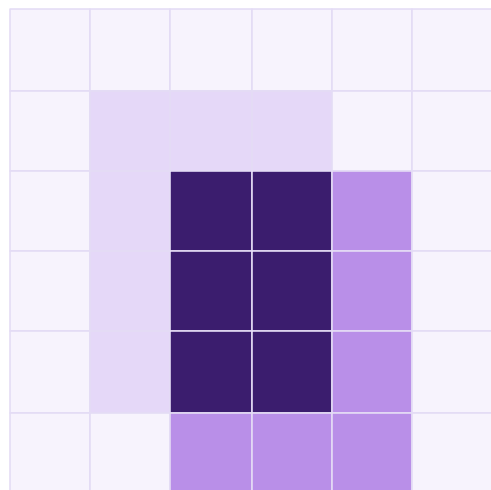
Panoptic = stuff + things

- Stuff → Road, Sky, Vegetation
- Things → Car 1, Car 2, Bus 1

Measuring segmentation quality

Segmentation needs pixel- and region-level metrics — not just bounding-box accuracy.

Mask IoU, pixel by pixel



ground truth
 prediction
 overlap

$$\text{IoU} = \text{overlap} / \text{union}$$



Pixel accuracy

Fraction of correctly labelled pixels. Can look high when background dominates.



Mean IoU (mIoU)

Average IoU across classes. The standard semantic-segmentation metric.



Dice / F1

$2 \times \text{overlap} \div (\text{ground truth} + \text{prediction})$.
Useful for small regions.

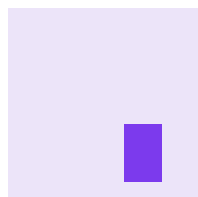


Panoptic Quality (PQ)

Segmentation quality $\times$ recognition quality —
for panoptic segmentation.

When evaluation metrics mislead you

A single score can hide important failure modes. Know the traps before trusting the number.



95% bg · 5% object

Class imbalance

95% accuracy means little when 95% of pixels are background. Report per-class metrics and mIoU.



0.5 easy 0.75 strict

IoU threshold choice

mAP@0.5 is more forgiving than mAP@0.5:0.95. Always state the threshold used.

precision ↑



threshold

recall ↓

Confidence threshold

Raising the threshold usually increases precision but reduces recall. Report the operating point.



1 blob

2 objects

Pixel vs object level

High pixel accuracy can hide merged or missed objects. Use object-level metrics such as PQ or instance mAP.

From zero to predictions: the code

```
detect_and_segment.py

from ultralytics import YOLO

# 1) Detection — start from COCO weights
model = YOLO("yolo11n.pt")
model.train(data="data.yaml",
            epochs=100, imgsz=640)

# 2) Instance segmentation — same API
seg = YOLO("yolo11n-seg.pt")
seg.train(data="data.yaml", epochs=100)

# 3) Predict, evaluate, export
pred = model.predict("street.jpg")
metrics = model.val() # mAP (mean Avg Precision) + masks
model.export(format="onnx")
```

One API, many tasks

Detect, segment, pose and classify all share the same train / val / predict workflow.

Always start pretrained

Point data.yaml at your images + labels and fine-tune COCO weights — not from scratch.

Validate honestly

model.val() reports box mAP and mask mAP; inspect per-class, not just the headline.

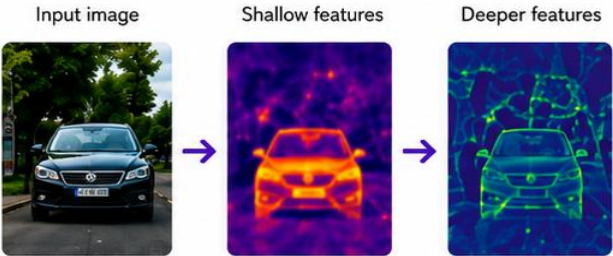
Export to deploy

One call to ONNX or TensorRT gets you to the edge.

What the outputs actually look like

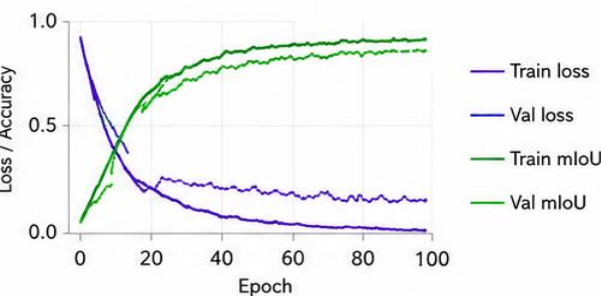
Self-explanatory visuals are how you communicate a vision model — six you'll produce again and again.

1 Feature maps



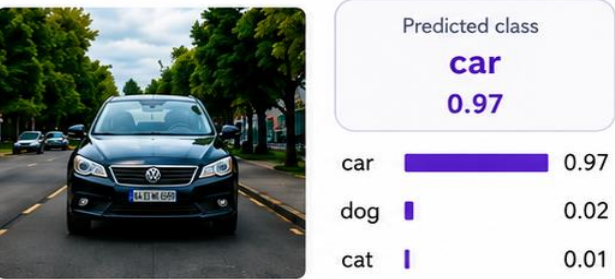
What the network sees at different depths.

2 Training curves



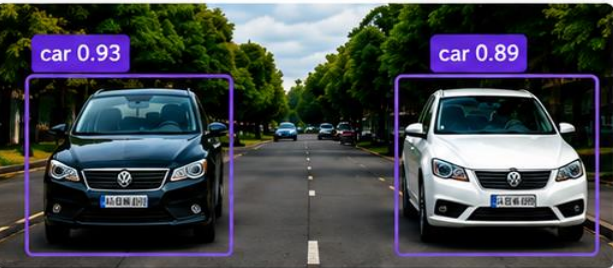
How the model learns over time.

3 Class predictions



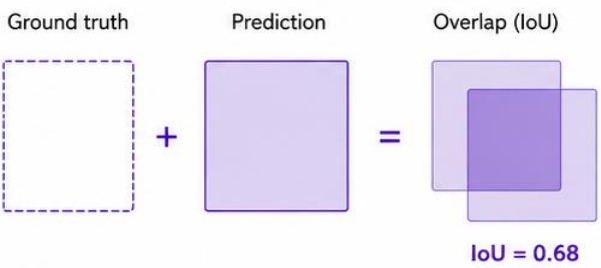
What class the model thinks it is.

4 Bounding boxes



Where objects are, with confidence.

5 IoU overlap



How much the prediction overlaps the truth.

6 Segmentation masks

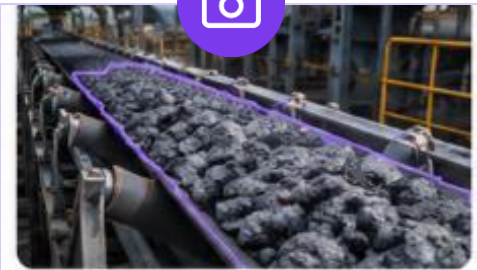
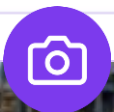


Per-pixel predictions of what everything is.

A Vedanta-style mineral-processing plant



The gap: ore grade, froth condition and equipment health on the line are still judged by eye — slow, inconsistent, and unsafe to inspect up close. Cameras already watch the plant; the data is unused.



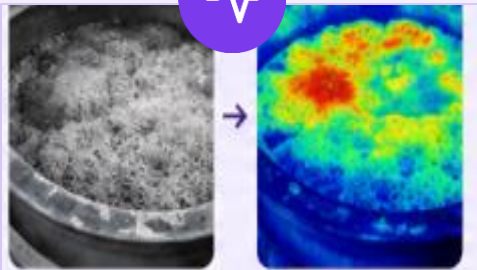
Conveyor-belt ore

Estimate size distribution and flag oversize rocks as they pass.

Size

% Oversize

Throughput



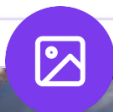
Froth in flotation

Read bubble texture to infer recovery and dose reagent correctly.

Recovery

Stability

Reagent dose



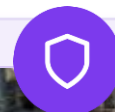
Equipment inspection

Spot worn liners, belt tears and blockages before they fail.

Wear

Tears

Blockages



Plant safety

PPE compliance and intrusion into hazardous zones, in real time.

Compliance

Intrusion

Alerts

How each task solves a different part

The same plant footage feeds three models — each answering a different operational question.



Classification

Is this froth healthy or collapsing?



Froth status

Healthy



Confidence

92%



Tag each camera frame with a grade or pass/fail state.

→ triggers a reagent-dosing alert



Object Detection

Where are the oversize rocks & is PPE worn?



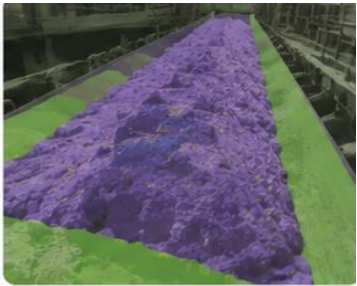
Draw boxes around rocks, workers, helmets and vehicles.

→ counts, locations, safety flags



Segmentation

What fraction of the belt is ore vs. gangue?



	Ore	68%
	Fines	24%
	Gangue	6%
	Backgrnd	2%



Per-pixel mask of ore, fines and background.

→ precise volume & grade estimate

Assumptions, Limitations & Business impact

Model performance depends on the data, environment and deployment context.

Assumptions & Limitations



DATA

Needs diverse, well-labelled data — annotation is the real cost.



ENVIRONMENT

Lighting, dust, camera angle and occlusion shift performance.



DEPLOYMENT

Edge compute limits model size and frame rate.



DRIFT

Plant changes can silently degrade model performance.

Business impact when it works



Higher throughput & recovery

Better process visibility and control.



Safer operations

Fewer people exposed to hazardous areas.



Less unplanned downtime

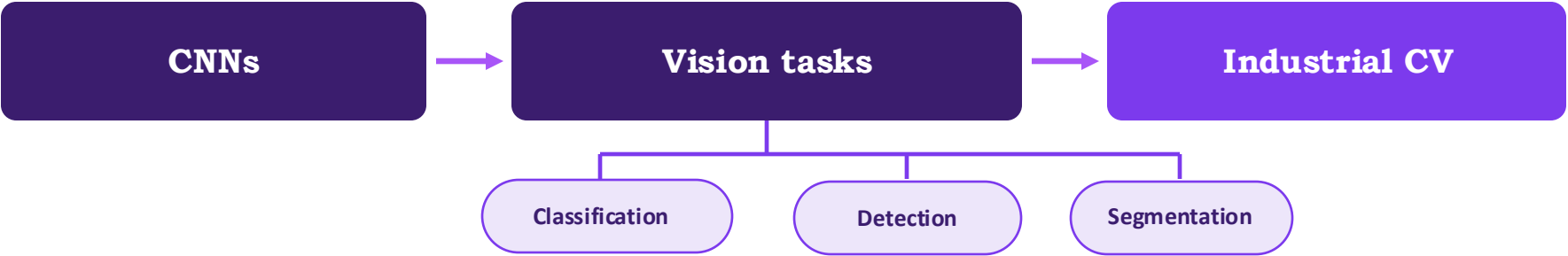
Earlier detection of equipment defects.



More consistent quality

More reliable product and process decisions.

Summary & Key Takeaways



Key idea: choose the task first, then the model and metric that match the business problem.

Where to start



Reuse, don't reinvent

Start with transfer learning and pretrained backbones.



Use proven models

Adapt YOLO, Faster R-CNN or U-Net instead of building from scratch.



Augment & regularise

Use augmentation, dropout and early stopping to reduce overfitting.



Measure the right thing

Track mAP for detection and mIoU / Dice for segmentation.

Thank you

